Stage 1 Step-by-Step Practical & Conceptual Recap With Steps Taken

1. Dependency Cleanup & Git Tracking (.gitignore & git rm)

- **What We Did:** Created a .gitignore file containing node_modules/ and ran:
  git rm -r --cached node_modules

- **Why We Did It:** In Stage 0, node_modules/ was staged into Git tracking. Running git rm -r --cached stripped thousands of third-party dependency files from Git's index while leaving the actual folder intact on disk.
- **Sysadmin Analogy:** Committing node_modules/ into Git is like backing up volatile cache or temp directories (/tmp/ or /var/cache/) inside a system configuration snapshot. Excluding dependencies keeps your repository lightweight, fast, and compliant with clean version control standards.

2. Express Routing Rules (app.get)

- **What We Built:** Added two GET routes to server.js:
  app.get('/', (req, res) => { ... });
  app.get('/health', (req, res) => { ... });

- **Sysadmin Analogy:** Express routing acts like an **Nginx Reverse Proxy or Firewall Access Control List (ACL)**. When an inbound TCP packet hits port 3000, the routing engine evaluates two header criteria:
  1. **HTTP Verb / Method:** GET (Data Retrieval)
  2. **URI Path:** / or /health

If a match is found, traffic routes to the designated request handler function. If no match exists, Express drops through to its default rule and returns an HTTP 404 Not Found response.

3. Shell Commands vs. Interpreter Syntax (Troubleshooting Step)

- **What Happened:** The terminal command git rm -r --cached node_modules was accidentally pasted onto line 11 inside server.js.
- **The Fix:** Removed the shell command from the JS script and executed it inside the PowerShell terminal prompt instead.
- **Sysadmin Analogy:** This is identical to pasting a Bash CLI command into an Nginx or Apache configuration file (nginx.conf). The service daemon fails to parse the file because

shell syntax is invalid inside an application script.

4. Memory Buffering vs Disk State (Ctrl + S & Daemon Restarts)

- **What Happened:** After editing server.js, running curl in the second terminal tab kept returning old Stage 0 data (Hello World!) and 404 Not Found for /health.
- **Why It Happened:**
  1. The code modifications were buffered in VS Code volatile RAM (indicated by the white dot next to server.js).
  2. The active Node process was still running the old bytecode loaded into RAM during startup.
- **The Fix:**
  1. Saved server.js (Ctrl + S) to write changes from RAM buffer to disk.
  2. Stopped the background daemon with Ctrl + C in the left terminal tab.
  3. Re-executed node server.js to force the runtime to read the fresh script from disk.
- **Sysadmin Analogy:** Editing a script without saving is like editing a file in vim without writing (:w). Restarting the daemon (Ctrl + C + node server.js) is equivalent to running systemctl restart myapp to reload new binary instructions into active system memory.

5. Structured JSON Serialization (res.json vs res.send)

- **What We Built:** Replaced plain text output with structured JSON payloads:
  res.json({ name: "Task API", version: "1.0", endpoints: ["/tasks"] });

- **Sysadmin Analogy:** Plain text (res.send) sends unstructured strings (Content-Type: text/html). res.json() automatically sets the MIME header to Content-Type: application/json; charset=utf-8.
- **Why It Matters:** Automated B2B integrations (such as invoice extraction pipelines for local Kenyan businesses) cannot parse raw text reliably. Structured JSON responses allow downstream database adapters and API clients to parse payloads into typed data fields safely.

6. Verification & Protocol Inspection (curl -i)

- **What We Executed:** Ran protocol inspection tests in the second terminal tab:
  curl -i http://localhost:3000/
  curl -i http://localhost:3000/health

- **Output Verified:**
  - GET /: Returned HTTP/1.1 200 OK, Content-Type: application/json, and body {"name":"Task API","version":"1.0","endpoints":["/tasks"]}.
  - GET /health: Returned HTTP/1.1 200 OK and body {"status":"ok"}.
- **Sysadmin Analogy:** GET / acts as an **SNMP Discovery Query or SSH System Banner**,

broadcasting API metadata. GET /health acts as a **System Heartbeat / ICMP Ping Probe** used by load balancers (like HAProxy or Nginx) to confirm service health.

7. Stage 1 Commit & Audit History

- **What We Executed:** Staged, committed, and audited Stage 1 in Git:
  git add .
  git commit -m "Stage 1: root and health endpoints"
  git log --oneline

- **Verified Log Output:**
  15fe97e (HEAD -> master) Stage 1: root and health endpoints
  f4a5e22 Stage 0: hello server

- **Sysadmin Analogy:** git add . stages changed configuration files into the staging area (like staging updates in a staging environment). git commit creates an immutable restore point (system snapshot), and git log reviews the system audit trail.